

DEVOPS & DEPLOYMENT

CLOUD & ON-PREMISE

RELEASE V2.4

OCPP Charge Point Management System

Complete Installation, Production Cloud VM Deployment,
PostgreSQL, Redis, Nginx WSS Proxy, PM2 & Hardware Setup Guide

TARGET AUDIENCE

DevOps Engineers, Cloud Architects & System Administrators

PUBLICATION DATE

August 2026

SUPPORTED OPERATING SYSTEMS

Ubuntu 24.04 / 22.04 LTS, Debian 12, GCP / AWS / Azure

PUBLISHER

The Charge Grid

Table of Contents

01. System Architecture & Infrastructure Sizing

02. Prerequisites & System Dependencies

03. Automated 1-Command Installer (install.sh)

04. Interactive Browser-Based Setup Wizard

05. Step-by-Step Local Development Setup

06. Production Ubuntu 24.04 VM Provisioning

07. PostgreSQL 15+ & Prisma Database Setup

08. Redis 7+ Caching & BullMQ Worker Queues

09. Nginx Reverse Proxy, TLS SSL & WSS Configuration

10. PM2 Process Management & Systemd Automation

11. Firewall (UFW) & Network Port Rules

12. Environment Variables Reference (.env)

13. Connecting Physical Chargers & Proxy Setup

14. Health Checks, Monitoring & Troubleshooting

1. System Architecture & Infrastructure Sizing

The **OCPP-CPMS** operates across a high-performance decoupled multi-tier architecture:

TIER	TECHNOLOGY	PORT	ROLE
Reverse Proxy	Nginx 1.24+ / Certbot	80 / 443	TLS termination, HTTP routing, WSS proxying
Frontend	Next.js 16 (React 19)	3002	Admin Dashboard & Mobile Companion UI
Backend API	Express 5 + TypeScript	3000	REST Endpoints, Auth, Roaming, Invoicing
OCPP Engine	Node.js ws (RFC 6455)	9220	Native OCPP 1.6-J & 2.1 WebSocket Server
Database	PostgreSQL 15+ / Prisma	5432	Relational transactional storage & telemetry
Cache & Queue	Redis 7+ / BullMQ	6379	Telemetry cache, pub/sub, background workers

Hardware Sizing Recommendations

FLEET SIZE	VCPU	RAM	STORAGE	CLOUD VM EXAMPLE
1 - 100 Chargers	2 vCPU	8 GB	50 GB NVMe	GCP e2-standard-2 / AWS t3.large
100 - 1,000 Chargers	4 vCPU	16 GB	200 GB NVMe	GCP e2-standard-4 / AWS c6i.xlarge
1,000 - 10,000 Chargers	8+ vCPU	32+ GB	500+ GB NVMe	GCP c2-standard-8 / AWS c6i.2xlarge

2. Automated 1-Command Installer & Setup Wizard

2.1 Automated One-Command Installer (`install.sh`)

Deploy the complete stack on Ubuntu 24.04 LTS with a single shell invocation:

```
sudo bash install.sh \  
  --frontend-domain "ui.yourdomain.com" \  
  --backend-domain "ocpp.yourdomain.com" \  
  -y
```

AUTOMATED TASKS EXECUTED

Installs Node.js 24, PostgreSQL, Redis, Nginx, Certbot, PM2; provisions database and schema; builds frontend; configures Nginx virtual hosts with WSS proxying; requests Let's Encrypt SSL; and registers systemd auto-start daemons.

2.2 Interactive Browser Setup Wizard (`interactive-setup.html`)

Open `interactive-setup.html` in any browser to configure domains, SMTP, database credentials, and Stripe/Mollie keys visually, then generate a custom deployment script.

3. Step-by-Step Manual Deployment Guide

3.1 PostgreSQL Database Provisioning

```
sudo apt update && sudo apt install -y postgresql postgresql-contrib
sudo systemctl enable postgresql && sudo systemctl start postgresql

sudo -u postgres psql <
```

3.2 Redis Cache Installation

```
sudo apt install -y redis-server
sudo systemctl enable redis-server && sudo systemctl start redis-server
redis-cli ping # Should return: PONG
```

3.3 Backend API & Prisma Setup

```
cd /var/www/ocpp-cpms/Backend
npm install --production=false
cp .env.example .env
# Edit .env to set DATABASE_URL, REDIS_URL, and JWT_SECRET
npx prisma generate
npx prisma db push --accept-data-loss
npm run create-superadmin -- "admin@thechargegrid.com" "SuperAdminPass2026!"
npx tsc
```

3.4 Frontend Next.js Production Build

```
cd /var/www/ocpp-cpms/Frontend
npm install
cat < .env.local
NEXT_PUBLIC_API_URL="https://ocpp.yourdomain.com/api"
EOT
npm run build
```

4. Nginx Reverse Proxy & WSS Configuration

Deploy the following virtual host configuration to `/etc/nginx/sites-available/ocpp-cpms` :

```
# 1. Frontend Next.js Admin Dashboard
server {
    server_name ui.yourdomain.com;
    location / {
        proxy_pass http://127.0.0.1:3002;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

# 2. Backend REST API & OCPP WSS Server
server {
    server_name ocpp.yourdomain.com;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # OCPP 1.6 and 2.1 WebSocket Server
    location ~ ^/OCPP/(1\.6|2\.1|2\.0\.1)/ {
        proxy_pass http://127.0.0.1:9220;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_read_timeout 86400s;
        proxy_send_timeout 86400s;
        proxy_buffering off;
    }
}
```

Enable the site and request automated TLS certificates:

```
sudo ln -s /etc/nginx/sites-available/ocpp-cpms /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
sudo certbot --nginx -d ui.yourdomain.com -d ocpp.yourdomain.com --non-interactive --agree-tos -m admin@y
```


